# RADAR STOCK

## Informe de decisiones de diseño y respuestas a las preguntas del equipo

| | |
|---|---|
| Proyecto | Aplicación móvil Android de inventario y ventas rápidas para almacenes de barrio, con landing de presentación |
| Asignatura | Taller de Proyectos de Especialidad (TPE058) - Ingeniería en Informática, AIEP |
| Contenido | Resumen de la conversación de trabajo del 1 y 2 de noviembre de 2026 |
| Estado | Borrador para validar con el docente. Reemplaza a la pauta técnica v0.1, que asumía otro stack |

En este informe, **(Oficial)** indica lo que dicen los documentos del proyecto (registro de requerimientos y Entregable Parte I, Avance III) y **(Propuesta)** indica una decisión de diseño que el documento no define y que el equipo debe confirmar.

## 1. Resumen de decisiones

| Tema | Decisión |
|---|---|
| Alcance | Requerimientos RF001-RF006 y RNF001-RNF005 del registro oficial. Fuera de alcance: facturación electrónica y pasarelas de pago. |
| App móvil | Android nativa en Kotlin con Jetpack Compose, desarrollada en Android Studio y entregada como APK descargable (RF006). Se asume que el docente espera un APK real; falta confirmarlo. |
| Backend | API en PHP con PDO (consultas preparadas), respuestas en JSON, alojada en Hostinger. |
| Base de datos | MySQL. Esquema *database/radar_stock_schema.sql* con tres ajustes (sección 5). |
| Landing | HTML, CSS y JavaScript, con botón de descarga del APK (RF005 y RF006). Se construye al final. |
| Registro de ventas | **Una sola forma:** pantalla rápida con productos como botones grandes (sección 4.4). |
| Escáner de cámara | Solo para agregar stock, crear productos y contar el inventario. **No se usa en la venta.** |
| Herramientas | Visual Studio Code, Android Studio, Antigravity (editor con agentes de IA), XAMPP para el entorno local, Git y GitHub. |

## 2. Requerimientos oficiales (referencia)

| ID | Nombre | Resumen |
|---|---|---|
| RF001 | Control y gestión de productos | Código de barras, nombre, categoría, precio, stock actual, stock mínimo y fecha de vencimiento. |
| RF002 | Ventas de salida rápida (POS) | Selección rápida de productos frecuentes, cantidades y descuento inmediato del stock. |
| RF003 | Alertas tempranas | Avisos por umbrales críticos: tasa de quiebre (TQ menor o igual a 25 %) y productos próximos a vencer. |
| RF004 | Control de accesos por roles | Administrador (inventario, reportes y configuración) y Vendedor/Colaborador (POS y consulta básica). |
| RF005 | Vitrina de valor (landing) | Página en HTML, CSS y JS con los tres pilares: control de mermas, agilidad en mostrador y prevención de desabastecimiento. |
| RF006 | Descarga directa del APK | Botón visible que descarga el APK alojado en Hostinger. |
| RNF001 | Usabilidad | Interfaz intuitiva, sin menús saturados ni capacitación prolongada. |
| RNF002 | Rendimiento | Consultas y transacciones en menos de 2 segundos (HTTP/HTTPS con respuesta JSON). |
| RNF003 | Capacidad de la base | Arquitectura relacional con índices B-Tree y consultas preparadas (PDO) sobre MySQL. |
| RNF004 | Seguridad | Contraseñas cifradas y protección contra inyección SQL (PDO). |
| RNF005 | Accesibilidad de la landing | Responsive y con carga en menos de 2 segundos. |

## 3. Respuestas a las preguntas del equipo

### P1. ¿Cuántos productos puedo almacenar?

Prácticamente no hay un límite para un almacén. MySQL maneja millones de filas, y un negocio de barrio suele tener entre 300 y 3.000 productos. El límite real es el espacio del plan de Hostinger. El RNF003 exige que el sistema no colapse al crecer, por lo que se usan índices y la lista se carga con filtros por categoría. **(Propuesta)** probar con 5.000 productos de ejemplo para demostrarlo en el informe.

### P2. ¿Cómo se ingresarán los productos?

El RF001 pide registrar, actualizar, consultar y categorizar. **(Propuesta)**:

- **Formulario en la app** con nombre, categoría, precio, código de barras, stock inicial, stock mínimo, stock deseado y fecha de vencimiento.
- **Escáner de cámara** para leer el código de barras, tanto al crear un producto como al reponerlo (flujos detallados en la sección 4).
- **Reposición:** acción «Agregar stock» en cada producto, que suma las unidades que llegaron.

### P3. ¿Cuáles serán los mensajes de alerta por baja de stock? (RF003)

**(Propuesta)** Las alertas aparecen en la pantalla de Alertas y como notificación del teléfono cuando se genera una nueva.

| Tipo | Mensaje |
|---|---|
| Stock bajo | Stock bajo: [producto] tiene [n] unidades (mínimo [m]). Conviene reponer. |
| Crítico | Stock crítico: [producto] está por agotarse ([n] unidades). |
| Agotado | Sin stock: [producto] se agotó. |
| Por vencer | Por vencer: [producto] vence en [d] días ([fecha]). |
| Vencido | Vencido: [producto] venció el [fecha]. Retíralo de la estantería. |

### P4. ¿Cuál es el porcentaje mínimo para alertar?

El **25 %** viene del RF003 (Oficial): la tasa TQ = (SA / SD) x 100, donde SA es el stock actual y SD el stock deseado, se considera crítica cuando es menor o igual a 25 %. Por eso se agrega la columna *stock_deseado* a la tabla de productos. Combinado con el stock mínimo por producto queda:

| Nivel | Condición |
|---|---|
| Crítico | Stock = 0, o stock actual menor o igual a 25 % del stock deseado |
| Bajo | Stock actual menor o igual al stock mínimo del producto (y no crítico) |
| Por vencer (propuesta) | Faltan 15 días o menos para la fecha de vencimiento |

### P5. ¿Cómo funciona la visualización de productos para registrar una venta? (RF002)

**Decisión del equipo:** una sola forma de registrar ventas, la pantalla rápida (detalle en la sección 4.4). Los productos se muestran como botones grandes con nombre y precio, ordenados de más vendido a menos vendido, con filtro por categoría. Cada toque suma una unidad; abajo se ve el resumen con el total y el botón «Confirmar venta», que descuenta el stock de inmediato.

**Ajuste necesario para que una sola forma funcione:** la pantalla debe poder mostrar **todos** los productos (no solo los 12 más vendidos). Si solo se mostraran los frecuentes, un producto nuevo, que aún no tiene ventas, no podría venderse nunca. Por eso se ordenan por ventas y se filtran por categoría.

### P6. ¿Cómo será el cálculo para cuadrar el inventario?

**(Propuesta)** El stock del sistema suma las reposiciones y resta las ventas. Para cuadrar, el administrador cuenta físicamente y la app calcula:

- **Cuadre de stock:** diferencia = stock contado - stock del sistema. Si el administrador confirma, el stock queda igual al contado (flujo en la sección 4.3).
- **Cuadre de caja:** diferencia = dinero contado - total vendido del día.

Con esto se mide el indicador de tiempo del proyecto, comparando el procedimiento en cuaderno con el de la app.

### P7. ¿Tendrá categorías como visualización?

Sí. El RF001 pide categorizar. Las categorías se verán como filtros en la lista de productos y en la pantalla de venta. **(Propuesta)** elegirlas desde una lista inicial (Bebidas, Lácteos, Abarrotes, Panadería, Aseo, Snacks) con opción de crear nuevas, para evitar escribirlas distinto cada vez.

### P8. ¿Habrá cierre de ventas diario, mensual o por rango? ¿Se puede editar?

El RF004 indica que el administrador accede a reportes. **(Propuesta)** pantalla «Resumen de ventas» donde el usuario elige **Hoy, 7 días, Este mes o un rango de fechas** y ve el total vendido, la cantidad de ventas y los productos más vendidos. No habrá un «cierre» que bloquee el día. Las ventas confirmadas no se editan, para proteger la integridad de los datos; los errores se corrigen con un ajuste de stock.

### P9 y P10. ¿Qué secciones se pueden editar y qué puede hacer cada perfil?

El RF004 (Oficial) define Administrador y Vendedor/Colaborador, este último con permisos restringidos al POS y a la consulta básica de existencias. Ambos roles de ese segundo grupo tienen los mismos permisos.

| Sección | Administrador | Vendedor / Colaborador |
|---|---|---|
| Venta rápida | Registrar | Registrar |
| Productos | Crear, editar y eliminar (solo si no tienen ventas) | Solo ver y filtrar |
| Precios, stock mínimo, stock deseado y vencimiento | Editar | Solo ver |
| Agregar stock, producto nuevo y conteo (escáner) | Sí | No |
| Alertas | Ver | Ver |
| Resumen de ventas | Ver | No |
| Usuarios y roles | Crear y editar | No |
| Mi cuenta (contraseña) | Editar | Editar |

## 4. Flujos de trabajo detallados

### 4.1 Agregar stock con escáner (administrador)

1. Tocar «Escanear» y apuntar la cámara al código del producto.
2. La app consulta a la API; si el código existe, muestra el nombre y el **stock actual** (por ejemplo, «Leche entera 1 L, stock actual: 6»).
3. Un campo numérico pregunta **«¿Cuántas unidades llegaron?»** (por ejemplo, 24).
4. Antes de guardar, la app muestra el resultado: «Stock nuevo: 30».
5. Al confirmar, el stock se actualiza y la app vuelve a la cámara para el siguiente producto.

Se escribe lo que llegó, no el total; la app hace la suma, para evitar errores de cálculo.

### 4.2 Producto nuevo (el código no existe)

1. Al escanear, la app avisa «Producto no registrado» y abre el formulario con el código ya escrito.
2. El administrador completa nombre, categoría, precio, stock mínimo, stock deseado y vencimiento.
3. En «Stock inicial» escribe las unidades que tiene; ese es el total de partida.
4. Al confirmar, el producto queda registrado.

El código de barras es solo un número: no contiene el nombre ni el precio, por eso la primera vez se completan los datos a mano. Los productos sin código (pan, fruta, granel) se registran escribiendo los datos sin escanear.

### 4.3 Cuadrar el inventario (conteo físico, administrador)

1. Escanear el producto y contar las unidades en la estantería.
2. Escribir el **total contado** (aquí sí es el total real).
3. La app muestra la diferencia, por ejemplo: «Sistema: 30, contado: 28, diferencia: -2».
4. Al confirmar, el stock queda en 28.

### 4.4 Venta rápida (única forma de registrar ventas)

1. Se abre la pantalla con los productos como botones grandes (nombre y precio), ordenados por más vendidos, y con filtros de categoría arriba.
2. Cada toque agrega 1 unidad a la venta. Los productos con stock 0 aparecen en gris y no se pueden agregar; los de stock bajo muestran una pequeña etiqueta de aviso.
3. Una barra fija inferior muestra el resumen: productos, cantidades (se ajustan con + y -) y el total.
4. Con «Confirmar venta», el servidor guarda la venta y su detalle y descuenta el stock en una sola operación. Si falta stock, avisa y no registra nada.

Reglas generales de ingreso de números: solo enteros mayores que cero (en el conteo se acepta cero); botón «Cancelar» que vuelve a la cámara sin guardar; mensaje corto de confirmación tras cada operación.

## 5. Revisión del esquema MySQL

El archivo *database/radar_stock_schema.sql* está en el motor correcto (MySQL, según el documento oficial) y cubre RF001 (productos), RF002 (ventas y detalle de ventas) y RF004 (usuarios con rol). La contraseña se guarda cifrada, como exige el RNF004. Ajustes pendientes:

1. **Columna stock_deseado** (INT) en *productos*, necesaria para calcular la tasa TQ del RF003.
2. **Índices (RNF003)** en productos(nombre_producto), productos(fecha_vencimiento) y ventas(fecha_venta).
3. **Compatibilidad con Hostinger:** en hosting compartido normalmente no se puede ejecutar CREATE DATABASE ni USE. Se crea la base desde el panel (hPanel) y se importa el script por phpMyAdmin, sin esas dos líneas.

Se descartó, por estar fuera del alcance oficial, agregar tablas de comercios, movimientos de stock y auditorías. Si más adelante se quiere guardar el historial de ajustes, sería un cambio pequeño.

## 6. Plan por partes alineado a la Carta Gantt

Según lo que se alcanza a leer de la Carta Gantt (verificar contra el archivo original), la arquitectura se define del 3 al 9 de noviembre, el desarrollo va del 10 al 27 de noviembre y las pruebas internas parten el 26 de noviembre. Son unas dos semanas y media de desarrollo, por lo que el alcance se mantiene estricto en RF001-RF004.

| Parte | Contenido | Requerimiento | Ventana de la Gantt |
|---|---|---|---|
| 1 | Base técnica: base de datos local, API de prueba y una pantalla Android que lista productos | RNF002, RNF003 | 3 a 14 nov (arquitectura e infraestructura) |

| Parte | Contenido | Requerimiento | Ventana de la Gantt |
|---|---|---|---|
| 2 | Login y roles (Administrador y Vendedor/Colaborador) | RF004, RNF004 | 15 a 23 nov (autenticación) |
| 3 | Productos: formulario, edición y luego escáner para agregar stock, producto nuevo y conteo | RF001 | 13 a 27 nov (funcionalidades) |
| 4 | Venta rápida (botones, resumen y descuento de stock) | RF002, RNF001 | 13 a 27 nov (funcionalidades) |
| 5 | Alertas y notificaciones | RF003 | 19 a 27 nov (alertas) |
| 6 | APK firmado, subida a Hostinger y landing con botón de descarga | RF005, RF006, RNF005 | Al cerrar el desarrollo, antes de la beta |

Orden de seguridad: primero lo obligatorio (partes 1 a 5 sin escáner). Si el tiempo se acorta, el escáner es lo primero que se posterga, porque el ingreso manual ya cumple el RF001.

## 7. Diferencias entre el Entregable y el registro de requerimientos

El Entregable (Avances 1 a 3) se redactó antes de cerrar los requerimientos y, por el espacio limitado de cada informe, los requerimientos finales quedaron en el registro de requerimientos funcionales y no funcionales. Por eso **el registro es la versión vigente y prevalece** sobre el texto del Entregable. Para la entrega final se recomienda alinear los puntos siguientes y dejar constancia en el «Registro de control de cambios» del Entregable.

- **Sección 3 del Entregable:** lista solo RF001-RF003 y RNF001-RNF003, pero el registro oficial tiene RF001-RF006 y RNF001-RNF005. Además, los nombres de RF001 y RF003 no coinciden entre ambos documentos.
- **Objetivo general:** habla de «aplicación web y móvil», pero según los requerimientos la web es solo la landing. Se sugiere: «sitio web de presentación y aplicación móvil Android».
- **Indicador del 20 %:** el objetivo específico 2 se refiere al tiempo de registro de ventas y la sección de impacto al tiempo de auditoría y cuadratura. Conviene unificarlo y definir cómo se medirá.
- **Carta Gantt:** incluye «lanzamiento en tiendas de aplicaciones», pero el RF006 establece descarga directa del APK y los límites indican entrega de un prototipo funcional.
- **Tecnologías:** React aparece en el FODA y en las herramientas, pero los requerimientos no lo usan. Los recursos mencionan WordPress con un plugin de IA, mientras que el RF005 pide una landing en HTML, CSS y JS. Hasta donde se sabe, codi.link es un editor de código en línea, no una herramienta de base de datos.
- **Fórmula de TQ:** (SA / SD) x 100 mide el nivel de stock respecto del deseado; conviene explicarla así en el informe para que no se confunda con una tasa de quiebres ocurridos.

## 8. Cambios de rumbo durante la conversación

Mientras no se tuvo el documento oficial, las primeras recomendaciones asumieron React, Node.js y SQL Server. Al revisar los documentos se corrigió lo siguiente:

- Se retira la recomendación de convertir el esquema a SQL Server: el motor oficial es MySQL con PDO.
- Se retira la idea de una PWA como app móvil: el RF006 exige un APK compilado desde Android Studio.

- Se retiran las tablas extra propuestas (comercios, movimientos, auditorías): fuera del alcance oficial.
- La pauta técnica v0.1 y los primeros prompts quedan reemplazados por este informe y por los nuevos prompts (sección 10).

## 9. Pendientes por confirmar

1. **Con el docente:** que el prototipo se entregue como APK nativo descargable, y que el escáner y el resumen de ventas (extras de bajo costo) sean aceptables dentro del alcance.
2. **Con el equipo:** definir cómo se medirá el indicador de reducción del 20 % y qué pruebas se harán con comerciantes.
3. **En el documento:** alinear el Entregable y la Carta Gantt con el registro de requerimientos (sección 7) y anotarlo en el registro de control de cambios.
4. **Técnico:** aplicar los tres ajustes al esquema (sección 5) y confirmar el plan de Hostinger para la API, la base y el alojamiento del APK.
5. **Siguiente paso de trabajo:** usar los prompts de la sección 10 en Antigravity, comenzando por el prompt de contexto y la Parte 1.

## 10. Prompts para iniciar el desarrollo

Cómo usarlos en Antigravity:

1. Pega el **Prompt 0** una sola vez al iniciar la sesión. Si más adelante se pierde el contexto, vuelve a pegarlo.
2. Pega los prompts de la **Parte 1** en orden (1A, 1B, 1C y 1D), **uno por vez**.
3. Prueba cada resultado antes de pasar al siguiente. Si algo falla, pega el mensaje de error completo y pide que lo explique.
4. Lee lo que el agente cambió antes de aceptarlo y haz un commit por cada tarea terminada.

### Prompt 0: contexto y reglas de negocio

```
Estoy construyendo «Radar Stock», una app Android de inventario y ventas rápidas para almacenes de barrio (proyecto de especialidad). Soy estudiante de segundo año y estoy aprendiendo.

STACK FIJO (no agregues otras tecnologías):
- App: Android nativa en Kotlin con Jetpack Compose, en Android Studio. Se entrega como APK.
- Backend: API en PHP con PDO que responde siempre en JSON. Se aloja en Hostinger.
- Base de datos: MySQL. El esquema es database/radar_stock_schema.sql y es la única fuente de verdad para nombres de tablas y columnas.
- Landing: HTML, CSS y JS (se hace al final).
- Carpetas del repo: database/, backend/, android/ y landing/.

REQUERIMIENTOS (alcance cerrado):
- RF001 productos: código de barras, nombre, categoría, precio, stock actual, stock mínimo, stock deseado y fecha de vencimiento.
- RF002 venta rápida. RF003 alertas. RF004 roles.
- Fuera de alcance: facturación electrónica y pasarelas de pago.
- Las respuestas de la API deben tardar menos de 2 segundos.

REGLAS DE NEGOCIO:
1. Roles: Administrador (todo) y Vendedor/Colaborador (solo registrar ventas, ver productos y alertas, y cambiar su propia contraseña).
2. Venta: UNA sola forma de registrarla. Pantalla con todos los productos como botones grandes (nombre y precio), ordenados de más a menos vendido, con filtro por categoría. Cada toque suma 1 unidad. Los productos con stock 0 aparecen en gris y no se pueden agregar. Una barra inferior muestra el resumen (cantidades con + y -) y el total. «Confirmar venta» guarda la venta y su detalle y descuenta el stock en una sola transacción; si falta stock, avisa y no registra nada. Las ventas confirmadas no se editan. La venta NO usa escáner.
3. Escáner de cámara (CameraX + ML Kit), solo para el administrador:
  - Agregar stock: escanear, mostrar nombre y stock actual, preguntar «¿Cuántas unidades llegaron?», mostrar «Stock nuevo: X», confirmar y volver a la cámara.
  - Producto nuevo: si el código no existe, abrir el formulario con el código ya escrito; «Stock inicial» es el total de partida.
  - Conteo: escribir el total contado, mostrar la diferencia (contado - sistema) y, al confirmar, dejar el stock igual al contado.
  - Solo enteros mayores que cero (en el conteo se acepta cero). Botón «Cancelar» que no guarda nada.
4. Alertas (pantalla de Alertas y notificación del teléfono): Crítico si stock = 0 o stock actual menor o igual al 25 % del stock deseado; Bajo si stock actual menor o igual al stock mínimo; Por vencer si faltan 15 días o menos; Vencido si ya pasó la fecha. Mensajes: «Stock bajo: [producto] tiene [n] unidades (mínimo [m]). Conviene reponer.», «Stock crítico: [producto] está por agotarse ([n] unidades).», «Sin stock: [producto] se agotó.», «Por vencer: [producto] vence en [d] días ([fecha]).», «Vencido: [producto] venció el [fecha]. Retíralo de la estantería.»
5. Resumen de ventas (solo administrador): Hoy, 7 días, Este mes o rango de fechas; muestra total vendido, número de ventas y productos más vendidos.
6. Categorías: se eligen de una lista (Bebidas, Lácteos, Abarrotes, Panadería, Aseo, Snacks) con opción de crear nuevas.

SEGURIDAD: todas las consultas SQL con PDO y sentencias preparadas; contraseñas con password_hash; nunca contraseñas en el código (configuración fuera de Git y un archivo de ejemplo).

REGLAS DE TRABAJO:
1. Haz solo lo que te pida en cada mensaje, en pasos pequeños.
2. Antes de ejecutar un comando, explícame en una línea para qué sirve.
3. No instales librerías que no te haya pedido.
4. Al terminar cada tarea, dime cómo probarla y espera mi confirmación antes de seguir.
5. Si algo no está claro, pregúntame en vez de asumir.
```

# Parte 1: de la base de datos a la pantalla

## Prompt 1A: base de datos

```
Edita database/radar_stock_schema.sql:
- Quita las líneas CREATE DATABASE y USE, para poder importarlo en una base ya creada (en
Hostinger no se pueden crear bases por script).
- Agrega la columna stock_deseado INT a productos.
- Agrega índices en productos(nombre_producto), productos(fecha_vencimiento) y
ventas(fecha_venta).
Crea además database/02_datos_prueba.sql con 10 productos de almacén de ejemplo, usando las
categorías Bebidas, Lácteos, Abarrotes, Panadería, Aseo y Snacks, e incluyendo algunos con stock
bajo el mínimo y otros que vencen en menos de 15 días.
No ejecutes nada. Explícame cómo crear la base radar_stock_db en phpMyAdmin con XAMPP, importar
ambos archivos y comprobar que quedó bien.
```

## Prompt 1B: API en PHP

```
En backend/ crea una API en PHP con PDO que:
- Tenga un archivo de configuración config.php (fuera de Git) con los datos de conexión, y
config.example.php versionado. Conexión con utf8mb4 y errores como excepciones.
- GET api/salud.php: ejecuta SELECT 1 y responde JSON {"estado": "ok"}, o un error claro en JSON
si falla la conexión.
- GET api/productos.php: devuelve en JSON la lista de productos (id, nombre_producto, categoria,
stock_actual, stock_minimo, precio_venta, fecha_vencimiento).
Yo uso XAMPP: explícame dónde colocar o enlazar la carpeta backend para que Apache la sirva, y
cómo probar ambas rutas en el navegador.
```

## Prompt 1C: app Android

```
En android/ crea un proyecto de Android Studio en Kotlin con Jetpack Compose y una pantalla
«Productos» que:
- Llame a api/productos.php con Retrofit y muestre la lista (nombre, stock y precio), con
indicador de carga y un mensaje claro si falla la conexión.
- Use como dirección base el servidor local desde el emulador (http://10.0.2.2/...) y permita HTTP
sin cifrar solo en modo debug.
Explícame cada archivo en pocas líneas y cómo ejecutar la app en el emulador.
```

## Prompt 1D: verificar y guardar

```
Revisa que todo funcione junto y dime exactamente qué debo ver en el emulador. Revisa los
.gitignore de la raíz y de android/ para que no se suban config.php, local.properties ni la
carpeta build. Prepárame un commit con un mensaje claro en español y no hagas push hasta que yo te
confirme.
```

Al terminar la Parte 1, el siguiente paso es la Parte 2 (login y roles). Cuando llegues ahí, se prepara el prompt correspondiente con lo que haya quedado construido.